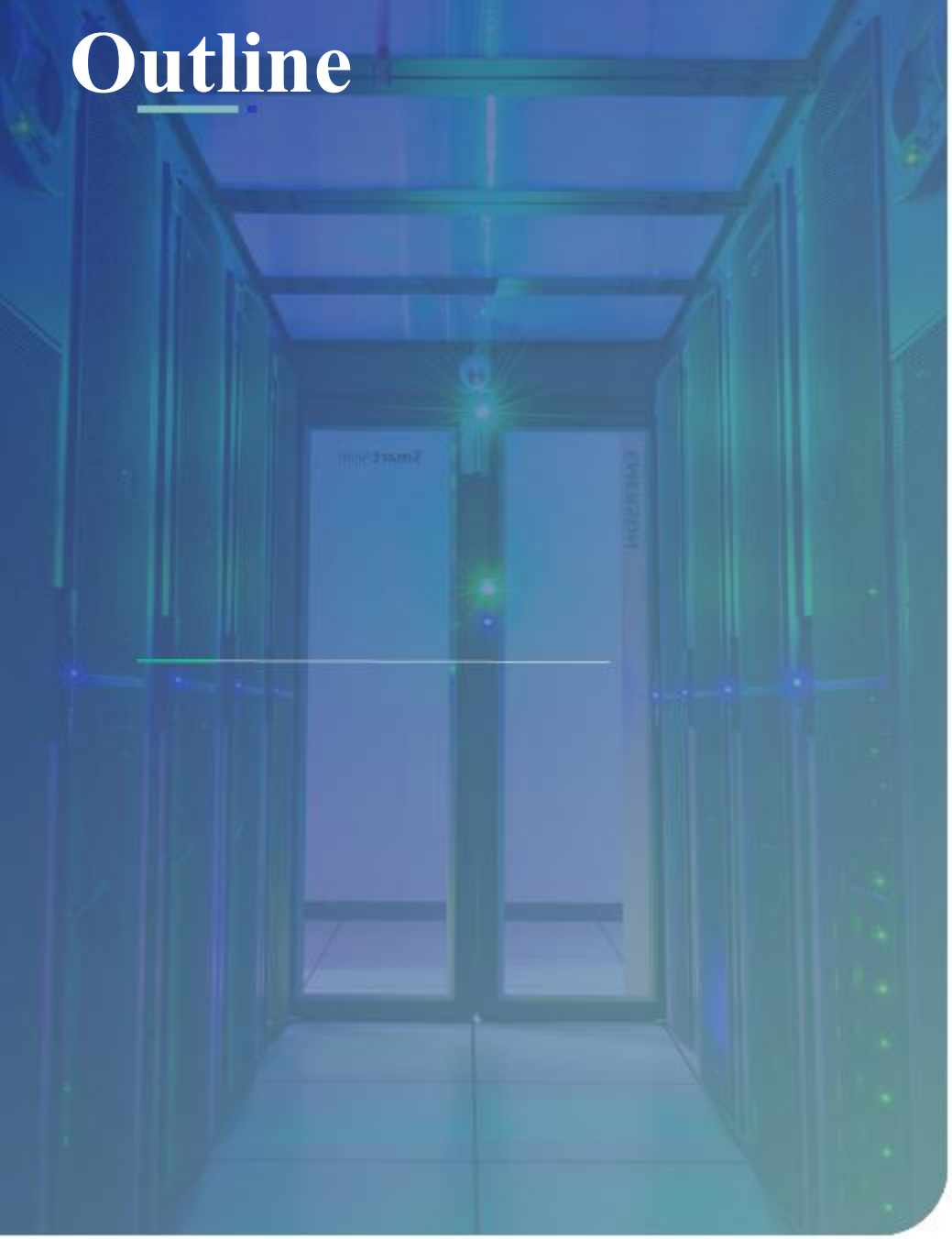
NSTDA Supercomputer Center
ThaiSC

Using AI modules on LANTA

Yutthana Wongnongwa, Ph.D.

NSTDA Supercomputer Center (ThaiSC)
National Electronics and Computer Technology Center (NECTEC)
National Science and Technology Development Agency (NSTDA)

Outline



- Basic knowledge of LANTA
  - System overview
  - Storage quota
  - Basic Lmod commands
  - Overview of Slurm job script
  - Checking job status
  - Cancel job
  - Check your billing account
  - File and folder permissions
- Using Miniforge3 module on LANTA
- Example of running Jupyter Notebook
- Example of Multi-GPUs/Multi-nodes training
- Using cray-python module on LANTA
- Using Apptainer module on LANTA

Basic knowledge of LANTA

How to Login to LANTA

Windows

✓ **Login to LANTA with MobaXterm**

https://youtu.be/NW40sB5W1dc

✓ **(Optional) Login to LANTA with VSCode**

https://youtu.be/4pzezkQZO3g

MacOS

✓ **Login to LANTA with Terminal**

https://youtu.be/kMTRAw-QHro

✓ **(Optional) Login to LANTA with Termius**

https://youtu.be/INdVbsNOS6c

✓ **(Optional) Login to LANTA with VSCode**

https://youtu.be/icueMyxhTZQ

LANTA frontend policy

lanta.nstda.or.th

- ✓ Login to LANTA
- ✓ Use general commands
- ✓ Submit Job
- ✓ Internet Speed**:** **up to 800 Mbps**

Note: Do not use for file transfer and run heavy process (>90%)

transfer.lanta.nstda.or.th

- ✓ File transfer
- ✓ Create conda environment
- ✓ Download container file
- ✓ Internet Speed: **up to 20 Gbps**

Note: Do not run heavy process (>90%)

System overview

Memory Nodes
1280 Cores

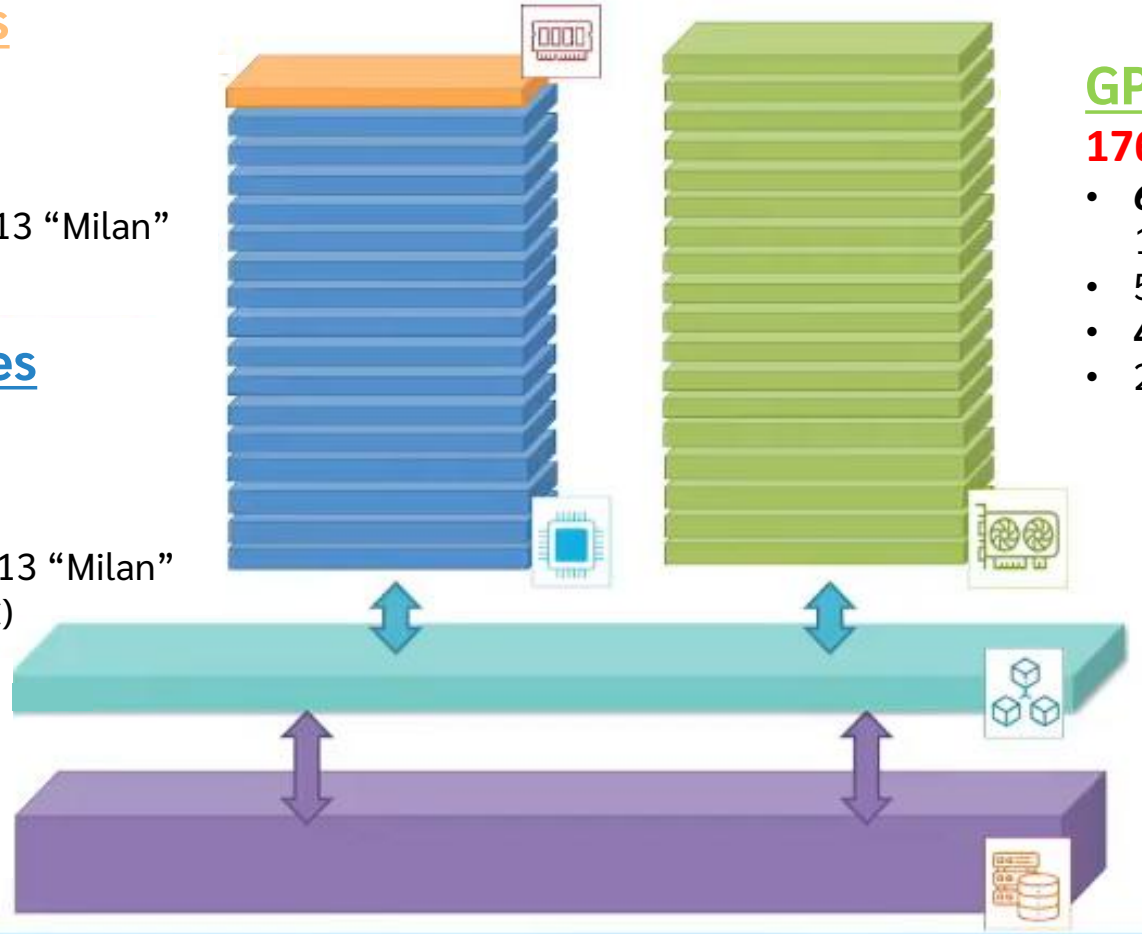
10x Compute nodes

- **128 cores/node**
  2 x AMD EPYC™ 7713 “Milan”
- 4 TB DRAM (ECC)

Compute Nodes
20,480 Cores

160x Compute nodes

- **128 cores/node**
  2 x AMD EPYC™ 7713 “Milan”
- 256 GB DRAM (ECC)



GPU Nodes 704 GPU

176x GPU nodes

- **64 cores/node**
  1 x AMD EPYC™ 7713 “Milan”
- 512 GB DRAM
- **4x Nvidia A100 40GB (HGX)**
- 2.4 TB Aggregated Bandwidth

High Performance Storage And Network

- 200 Gbps
- 10 PB HPC storage
- Connected all compute and storage via dragonfly network

https://thaisc.io/lanta/

Check LANTA status

Execute “**sinfo**” command to check LANTA status

```
PARTITION     AVAIL  TIMELIMIT  NODES  STATE NODELIST
compute*         up 5-00:00:00     46    mix lanta-c-[004-006,010,014-016,020,
compute*         up 5-00:00:00     24  alloc lanta-c-[001-003,007-009,011-013,
compute*         up 5-00:00:00     88   idle lanta-c-[022-064,114-158]
compute-devel    up    2:00:00      2   idle lanta-c-[159-160]
gpu              up 5-00:00:00      5    mix lanta-g-[001-002,005-007]
gpu              up 5-00:00:00     10  alloc lanta-g-[003-004,009-016]
gpu              up 5-00:00:00    159   idle lanta-g-[008,017-174]
gpu-devel        up    2:00:00      2   idle lanta-g-[175-176]
memory           up 5-00:00:00      4    mix lanta-m-[001-004]
memory           up 5-00:00:00      5  alloc lanta-m-[005-009]
memory           up 5-00:00:00      1   idle lanta-m-010
```

State	Detail
idle	The whole machine is available to use.
alloc	The whole machine is currently in use.
mix	Some resources of the machine are available to use, while some are already in use.
down	The machine is NOT available for use.
drain	The machine is NOT available for further use, due to some minor issues with the system.

LANTA storage quota

Path	Storage space	Inode/count	Permission
/home/username	100 GB	600,000	For each user account
/project/zz991000-zdeva/zz991000	200 TB	2,000,000,000	Share among all project members
/project/zz991000-zdeva/zz991xxx	200 TB	2,000,000,000	Share among all project members

*** Important***

To check remaining quota, execute “**myquota**” command

```
                              Block Quota Limits          Inode Quota Limits
                                 Used       Size          IUsed       Inodes
                          ----------------------   ----------------------
/home/hpcuser                  31.57G       100G         172925       600000
/project/lt200199-espre        61.81G         5T         192973     50000000
```

Basic Lmod commands

Command	Subcommand	<Arg(0) ...>	= action
module	overview		= list all available modules by short names and count of versions
module	help	*name*	= print short detail of *name*
module	spider	*name*	= print detailed information of *name* (slow)
module	avail	*name*	= list all available versions of *name*; if none specified, then list all
module	list		= list all modules currently in use
module	load	*name/version*	= load *name/version* module
module	unload	*name/version*	= unload *name/version* module
module	swap	*old new*	= replace *old* with *new*
module	purge		= unload all modules
module	reset		= restore to the default set of modules (LANTA: PrgEnv-cray)

*** Tips ***

Use “ml” instead “module”

and “module list”

Overview of Slurm job script

Slurm job scripts enable the execution of software on hardware resources for a specified duration

Example: job script

```
#!/bin/bash
#SBATCH --partition=partition_name
#SBATCH --nodes=num_nodes
#SBATCH --ntasks-per-node=ntask/node
#SBATCH --gpus-per-node=ngpu/node
#SBATCH --time=D-HH:MM:SS
#SBATCH --job-name=jobname
#SBATCH --account=ltxxxxxx

module purge
module load libs-or-progs

srun my_hpc_application
```

1) #SBATCH directive/header

- State required resources
- Various possible approaches/combinations, see sbatch

2) Instruction/Script

- Run on the first scheduled node
  - Load/Set software environment
  - Execute instructions
- Use srun to launch software processes across nodes
- By default, **srun inherits most options from sbatch**; since all allocated resources should be utilized.

Full detail: https://slurm.schedmd.com/sbatch.html, https://slurm.schedmd.com/srun.html

Check job status

Execute "**myqueue**" command to check your job status

```
username@x3002c0s7b0n0:~> myqueue
JOBID    PARTITION    NAME     USER     ST          TIME    NODES    NODELIST(REASON)
151602     compute    test4   username   PD          0:00      1       (Resources)
151600     compute    test3   username   PD          0:00      1       (Priority)
150500     compute    test3   username   PD          0:00      1       (QOSGrpBillingMinutes)
140215     compute    test2   username    R    03:25:08      1       lanta-c-055
149496     compute    test1   username    R 2-02:11:27      1       lanta-g-042
```

(REASON)

(Resources) = Not enough resources for your job. However, **your job is the 1st in the queue.**

(Priority) = Not enough resources for your job. **Your job is waiting in the queue.**

(QOSGrpBillingMinutes) = Remaining SHr of the project is NOT enough for the job to run until reaching its runtime limit.

Check by executing "sbalance" --> try reducing the runtime lime (-t) or top up more SHr

Cancel job

Execute “**scancel [JOBID]**” command to cancel job

```
username@x3002c0s7b0n0:~> myqueue
JOBID   PARTITION   NAME    USER    ST        TIME   NODES   NODELIST(REASON)
151602   compute   test4  username  PD        0:00     1      (Resources)
151600   compute   test3  username  PD        0:00     1      (Priority)
150500   compute   test3  username  PD        0:00     1      (QOSGrpBillingMinutes)
140215   compute   test2  username   R   03:25:08     1      lanta-c-055
149496   compute   test1  username   R 2-02:11:27     1      lanta-g-042
username@x3002c0s7b0n0:~> scancel 140215
username@x3002c0s7b0n0:~> myqueue
JOBID   PARTITION   NAME    USER    ST        TIME   NODES   NODELIST(REASON)
151602   compute   test4  username  PD        0:00     1      (Resources)
151600   compute   test3  username  PD        0:00     1      (Priority)
150500   compute   test3  username  PD        0:00     1      (QOSGrpBillingMinutes)
149496   compute   test1  username   R 2-02:11:27     1      lanta-g-042
```

Check your billing account

Execute "**sbalance**" command to check SHr of your billing accounts

```
username@x3002c0s7b0n0:~> sbalance

Account     Descr    Remaining(%)  Allocation(SHr)      Used(SHr)    Remaining(SHr)
-------------------------------------------------------------------------------------
lt200173    cperft         37.10%          4281.00        2692.94           1588.06
ltxxxxxx    yyyyy          40.16%        221875.00      132764.58          89110.42
```

- Use "**sbalance --detail**" to display (approximated) usage per users
- See "**sbalance --help**" for more available options

Billing of allocated resource

RAM* = Free RAM, readily for software to use

1 GiB = 1024 MiB

Whole-node allocation

	CPU cores	GPU cards	RAM* (GiB)	Cost per hour
lanta-c-xxx	128	-	up to 250G	1 SHr
lanta-g-xxx	64	4	up to 502G	3 SHr
lanta-m-xxx	128	-	up to 4018G	4 SHr

Standard allocatable pack of resources

	CPU cores	GPU cards	RAM* (MiB)	Cost per hour
lanta-c-xxx	1	-	1,900M	~0.0078 SHr
lanta-g-xxx	16	1	124,000M	0.75 SHr
lanta-m-xxx	1	-	32,000M	0.03125 SHr

*** Warning ***

This implies that

- 32 CPU + 2 GPU,
- 2 CPU + 2 GPU,
- 32 CPU + 1 GPU

equally cost 2 x 0.75 = 1.5 SHr per hour, because 2 packs (half GPU node) are retained for your job.

Note: 1) For lanta-g-xxx, every extra CPU core + 8 GiB RAM will cost 0.046875 SHr per hour, and then every further ~1,365 MiB RAM will cost ~0.0078 SHr per hour on top.

2) For lanta-m-xxx, every extra 8,000 MiB RAM will cost ~0.0078 SHr per hour on top.

File and folder permissions

Execute "ll" command to check your file and folder permissions

```
username@x3002c0s7b0n0:~> ll
drwxr-xr-x  11 username lt200xxx  4096 Apr 18 16:14 Your_Directory
```

Execute "**chmod g+w**" command to change your file and folder permissions

```
username@x3002c0s7b0n0:~> chmod g+w Your_Directory
username@x3002c0s7b0n0:~> ll
drwxrwxr-x  11 username lt200xxx  4096 Apr 18 16:14 Your_Directory
```

- u - The file owner.
- g - The users who are members of the group.
- o - All other users.

Permission	Character	Meaning on File
Read	-	The file is not readable. You cannot view the file contents.
	r	The file is readable.
Write	-	The file cannot be changed or modified.
	w	The file can be changed or modified.
Execute	-	The file cannot be executed.
	x	The file can be executed.

FAQ: Incorrect storage quota

- "myquota" will double count files in /project/ path, if their group is your username.
- This usually happens when `mv` command is used to move files between /project/ and /home/

```
skanoksi@x3002c0s9b0n0:/project/thaisc/skanoksi/WorkSpace/tutorial> ls -lR
.:
total 8
drwxr-sr-x 2 skanoksi thaisc 4096 [redacted] dirA
drwxr-xr-x 2 skanoksi thaisc 4096 [redacted] dirB

./dirA:
total 13778020
-rw-r--r-- 1 skanoksi thaisc 14108684400 [redacted] largefile

./dirB:
total 13762564
-rw-r--r-- 1 skanoksi skanoksi 14108684400 [redacted] largefile
```

Count in /project/ (5,120 GB)

Count in both /project/ (5,120 GB) and /home (100GB) !!

To fix this issue, use

1. **`chgrp -Rf ltxxxxxx your-directory`**
   = to change group of all files inside your-directory
2. **`find your-directory -type d -exec chmod g+s {} \;`**
   = to add setuid bit (+s) to group permission (g) of all directories (-type d) inside your-directory

Using Miniforge3 module on LANTA

Using Miniforge3 module on LANTA

NSTDA Supercomputer Center ThaiSC

➢ Load the Mamba module and activate your environment

```
username@lanta:~> ml Miniforge3/25.3.0-3
username@lanta:~> conda activate environment-name
(environment-name) username@lanta:~>
```

➢ View a list of your environments

```
username@lanta:~> conda env list
# conda environments:
#
base                  /lustrefs/disk/modules/easybuild/software/Miniforge3/25.3.0-3
netcdf-py39           /lustrefs/disk/modules/easybuild/software/Miniforge3/25.3.0-3/envs/netcdf-py39
pytorch-2.8.0         /lustrefs/disk/modules/easybuild/software/Miniforge3/25.3.0-3/envs/pytorch-2.8.0
tensorflow-2.20.0     /lustrefs/disk/modules/easybuild/software/Miniforge3/25.3.0-3/envs/tensorflow-2.20.0
```

Using Miniforge3 module on LANTA

➢ Deactivate your environment

```
(environment-name) username@lanta:~> conda deactivate
username@lanta:~>
```

➢ Unload the Miniforge3 module

```
username@lanta:~> module unload Miniforge3
```

➢ Unload all currently loaded modules

```
username@lanta:~> module purge
```

Note: Before you use the **module unload Miniforge3** or **module purge** command, you must **deactivate your environment** with the conda deactivate command.

Creating the environment in the user's home

NSTDA Supercomputer Center ThaiSC

➢ Create the environment

```
username@lanta:~> conda create -n myenv
.
.
proceed ([y]/n)?
```

➢ Create the environment with a specific version of packages

```
username@lanta:~> conda create -n myenv python=3.9
```

```
username@lanta:~> conda create -n myenv python=3.9 numpy=1.23.5
```

➢ Activate your environment

```
username@lanta:~> conda activate myenv
```

Creating the environment in the project's home

NSTDA Supercomputer Center ThaiSC

- **Specify a location for the environment**

```
username@lanta:~> conda create --prefix /your project directory/envs
```

- **Specify a location for the environment with a specific version of a packages**

```
username@lanta:~> conda create --prefix /your project directory/envs python=3.9 numpy=1.23.5
```

- **Activate your environment**

```
username@lanta:~> conda activate /your project directory/envs
```

Creating the environment from a requirements.yml file

NSTDA Supercomputer Center ThaiSC

➢ A simple requirements.yml file

```
name: myenv
dependencies:
  - python=3.9
  - numpy=1.23.5
  - pandas
```

➢ Create the environment from the requirements.yml file in the user's home

```
username@lanta:~> conda env create -f requirements.yml
```

➢ Create the environment from the requirements.yml file in the project's home

```
username@lanta:~> conda env create -f requirements.yml --prefix /your project directory/envs
```

FAQ: Using R language with Miniforge3

NSTDA Supercomputer Center ThaiSC

➢ Create the R environment in the user's home

```
username@lanta:~> conda create -n r_env r-essentials r-base
.
.
proceed ([y]/n)?
```

➢ Create the R environment in the project's home

```
username@lanta:~> conda create --prefix /your project directory/r_env
r-essentials r-base
.
.
proceed ([y]/n)?
```

➢ Activate your environment

```
username@lanta:~> conda activate r_env
username@lanta:~> conda activate /your project directory/r_env
```

Running python script via Miniforge3 module

NSTDA Supercomputer Center ThaiSC

➢ Running on Compute node

```
#!/bin/bash
#SBATCH -p compute                  # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 –c 128                 # Specify number of nodes and processors per task
#SBATCH --ntasks-per-node=1         # Specify tasks per node
#SBATCH -t 120:00:00                # Specify maximum time limit (hour: minute: second)
#SBATCH -A tb901xxx                 # Specify project name
#SBATCH -J JOBNAME                  # Specify job name

module load Miniforge3/25.3.0-3     # Load the Mamba module
conda activate tensorflow-2.20.0    # Activate your environment

python3 file.py                     # Run your program or executable code
```

Note: Full node: -c 128, Half node: -c 64, ¼ node: -c 32

Running python script via Miniforge3 module

NSTDA Supercomputer Center ThaiSC

➢ Running on GPU node

```
#!/bin/bash
#SBATCH -p gpu                          # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 –c 16                      # Specify number of nodes and processors per task
#SBATCH --gpus-per-node=4               # Specify number of GPU per task
#SBATCH --ntasks-per-node=4             # Specify tasks per node
#SBATCH -t 120:00:00                    # Specify maximum time limit (hour: minute: second)
#SBATCH -A tb901xxx                     # Specify project name
#SBATCH -J JOBNAME                      # Specify job name

module load Miniforge3/25.3.0-3         # Load the Mamba module
conda activate tensorflow-2.20.0        # Activate your environment

python3 file.py                         # Run your program or executable code
```

Note: 1 GPU card: --gpus-per-node=1, 2 GPU cards: --gpus-per-node=2, 4 GPU cards: --gpus-per-node=4

Example of running Jupyter Notebook

Example of running Jupyter Notebook

NSTDA Supercomputer Center ThaiSC

Using the example to run Jupyter Notebook

```
username@lanta:~> cd /project/your_project
username@lanta:~> cp -r /project/common/Miniforge3/ .
username@lanta:~> cd Mamba/
username@lanta:~> ls
DDP_1N  DDP_2N  Finetune_BERT_1N  Finetune_BERT_2N  Jupyter_GPU
```

Example of running Jupyter Notebook

NSTDA Supercomputer Center ThaiSC

```
port=$(shuf -i 6000-9999 -n 1)
USER=$(whoami)
node=$(hostname -s)

# jupyter notebookng instructions to the output file
echo -e "

   Jupyter server is running on: $(hostname)
   Job starts at: $(date)

   Copy/Paste the following command into your local terminal
   --------------------------------------------------------------------
   ssh -L $port:$node:$port $USER@lanta.nstda.or.th -i id_rsa
   --------------------------------------------------------------------
   Open a browser on your local machine with the following address
   --------------------------------------------------------------------
   http://localhost:${port}/?token=XXXXXXXX (see your token below)
   --------------------------------------------------------------------
   "

# start a cluster instance and launch the jupyter server

unset XDG_RUNTIME_DIR
if [ "$SLURM_JOBTMP" != "" ]; then
   export XDG_RUNTIME_DIR=$SLURM_JOBTMP
fi

jupyter notebook --no-browser --port $port --notebook-dir=$(pwd) --ip=$node
```

Running Jupyter Notebook

NSTDA Supercomputer Center ThaiSC

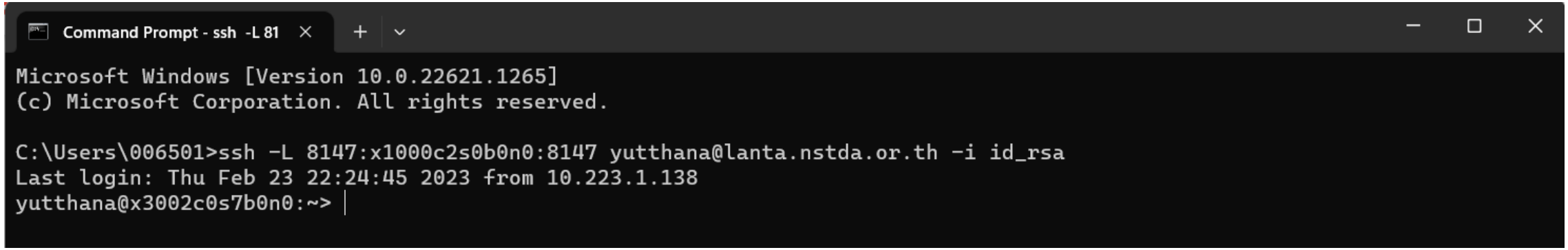
➢ **There are 3 steps to run Jupyter Notebook on LANTA**

1. Submit your job and read your slurm-xxxxx.out

```
username@lanta:~> sbatch script.sh
username@lanta:~> cat slurm-xxxxx.out
```

2. Copy/Paste the following command into your local terminal

```
ssh -L 7637:x1000c2s0b0n0:7637 username@lanta.nstda.or.th -i id_rsa
```



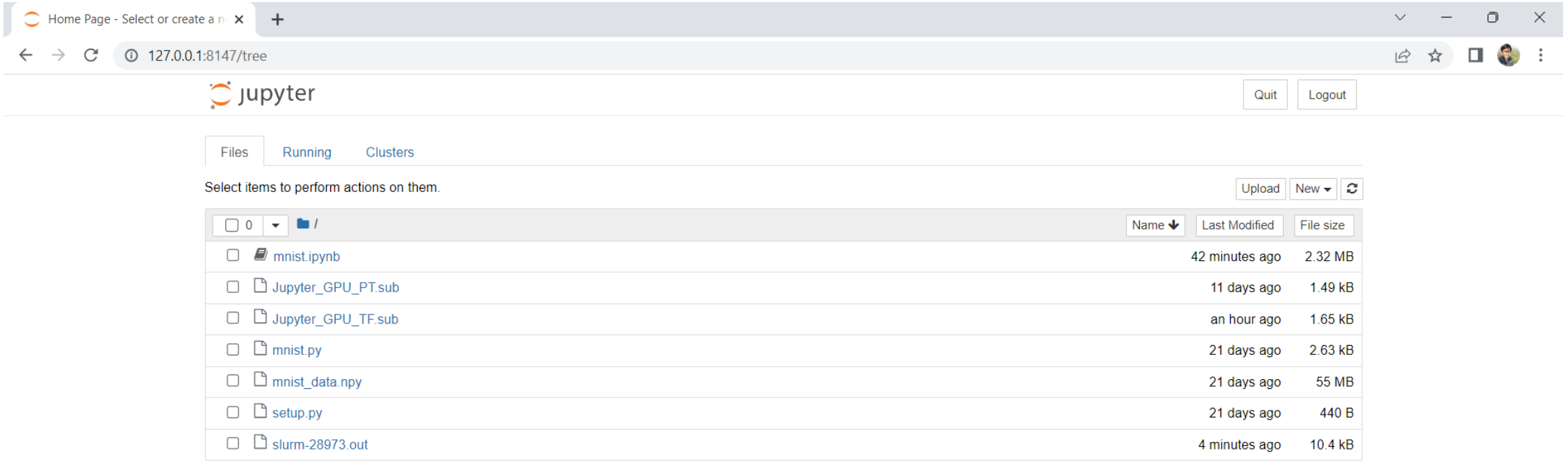
Running Jupyter Notebook

NSTDA Supercomputer Center ThaiSC

- **There are 3 steps to run Jupyter Notebook on LANTA**

3. Open a browser on your local machine with the following address (Final line in .out)

```
http://127.0.0.1:8147/?token=9720bf83310e36e6d6adb205ac3d2dc01c0518b73fb8421e
```



Example of Multi-GPUs/Multi-nodes training

Example of Multi-GPUs/Multi-nodes training

NSTDA Supercomputer Center ThaiSC

Using the example to run Multi-GPUs/Multi-nodes training

```
username@lanta:~> cd /project/your_project
username@lanta:~> cp -r /project/common/Miniforge3/ .
username@lanta:~> cd Mamba/
username@lanta:~> ls
DDP_1N  DDP_2N  Finetune_BERT_1N  Finetune_BERT_2N  Jupyter_GPU
```

Example of Multi-GPUs/Multi-nodes training

NSTDA Supercomputer Center ThaiSC

```
#!/bin/bash
#SBATCH -p gpu                  # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 -c 16              # Specify number of nodes and processors per task
#SBATCH --ntasks-per-node=4     # Specify number of tasks per node
#SBATCH --gpus-per-node=4       # Specify total number of GPUs
#SBATCH -t 2:00:00              # Specify maximum time limit (hour: minute: second)
#SBATCH -A your_project         # Specify project name
#SBATCH -J JOBNAME              # Specify job name

module load Miniforge3/25.3.0-3
conda activate your_environment

export HF_HOME=/project/your_project_directory/hf/misc
export HF_DATASETS_CACHE=/project/your_project_directory/hf/datasets
export TRANSFORMERS_CACHE=/project/your_project_directory/hf/models
export HF_DATASETS_OFFLINE=1
export TRANSFORMERS_OFFLINE=1

srun python script.py
```

Using cray-python module on LANTA

Using cray-python module on LANTA

NSTDA Supercomputer Center ThaiSC

➢ Load the cray-python module

```
username@lanta:~> ml av cray-python
---------------------------- /opt/cray/pe/lmod/modulefiles/core ----------------------------
cray-python/3.9.13.1    cray-python/3.10.10    cray-python/3.11.7 (D)

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".
username@lanta:~> ml cray-python/3.11.7
```

➢ Activate the virtual environment

```
username@lanta:~> source /path/to/virtual/environment/bin/activate
(env-name) username@lanta:~>
```

Note: For R language users, use the cray-R module instead.

Using cray-python module on LANTA

➢ Deactivate the virtual environment

```
(env-name) username@lanta:~> deactivate
username@lanta:~>
```

➢ Unload the cray-python module

```
username@lanta:~> module unload cray-python
```

Creating the virtual environment in the specific path

NSTDA Supercomputer Center ThaiSC

- **Create the virtual environment**

```
username@lanta:~> python -m venv ./myenv
```

- **Activate the virtual environment**

```
username@lanta:~> source ./myenv/bin/activate
(myenv) username@lanta:~>
```

- **Removal of the virtual environment**

```
username@lanta:~> rm -rf ./myenv
```

Running python script via cray-python module

NSTDA Supercomputer Center ThaiSC

➢ Running on Compute node

```
#!/bin/bash
#SBATCH -p compute                          # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 –c 128                         # Specify number of nodes and processors per task
#SBATCH --ntasks-per-node=1                 # Specify tasks per node
#SBATCH -t 120:00:00                        # Specify maximum time limit (hour: minute: second)
#SBATCH -A tb901xxx                         # Specify project name
#SBATCH -J JOBNAME                          # Specify job name

module load cray-python/3.11.7              # Load the crat-python module
source /home/username/myenv/bin/activate    # Activate your virtual environment

python3 file.py                             # Run your program or executable code
```

Note: Full node: -c 128, Half node: -c 64, ¼ node: -c 32

Running python script via cray-python module

NSTDA Supercomputer Center ThaiSC

➢ Running on GPU node

```
#!/bin/bash
#SBATCH -p gpu                          # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 –c 16                      # Specify number of nodes and processors per task
#SBATCH --gpus-per-node=4               # Specify number of GPU per task
#SBATCH --ntasks-per-node=4             # Specify tasks per node
#SBATCH -t 120:00:00                    # Specify maximum time limit (hour: minute: second)
#SBATCH -A tb901xxx                     # Specify project name
#SBATCH -J JOBNAME                      # Specify job name

module load cray-python/3.11.7          # Load the crat-python module
source /home/username/myenv/bin/activate      # Activate your virtual environment

python3 file.py                         # Run your program or executable code
```

Note: 1 GPU card: --gpus-per-node=1, 2 GPU cards: --gpus-per-node=2, 4 GPU cards: --gpus-per-node=4

Using Apptainer module on LANTA

Using Apptainer module on LANTA

➢ Load the Apptainer module

```
username@lanta:~> ml av Apptainer
---------------------------- /opt/cray/pe/lmod/modulefiles/core ----------------------------
Apptainer/1.1.6

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".
username@lanta:~> ml Apptainer/1.1.6
```

➢ Unload the cray-python module

```
username@lanta:~> module unload Apptainer
```

Using Apptainer module on LANTA

NSTDA Supercomputer Center ThaiSC

➢ Using the apptainer pull command

```
username@lanta:~> apptainer pull [pull options] [output file] <URI>
```

➢ Source

https://hub.docker.com/

https://catalog.ngc.nvidia.com/containers

➢ Examples

From Docker

```
username@lanta:~> apptainer pull tensorflow.sif docker://tensorflow/tensorflow:latest-gpu
```

From NVIDIA NGC Catalog

```
username@lanta:~> apptainer pull pytorch.sif docker://nvcr.io/nvidia/pytorch:24.05-py3
```

Running python script via Apptainer module

NSTDA Supercomputer Center ThaiSC

➢ Running on Compute node

```
#!/bin/bash
#SBATCH -p compute                      # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 –c 128                     # Specify number of nodes and processors per task
#SBATCH --ntasks-per-node=1             # Specify tasks per node
#SBATCH -t 120:00:00                    # Specify maximum time limit (hour: minute: second)
#SBATCH -A tb901xxx                     # Specify project name
#SBATCH -J JOBNAME                      # Specify job name

module load Apptainer/1.1.6             # Load the Apptainer module

apptainer exec -B $PWD:$PWD file.sif python3 file.py        # Run your program
```

Note: Full node: -c 128, Half node: -c 64, ¼ node: -c 32

Running python script via Apptainer module

NSTDA Supercomputer Center ThaiSC

➢ Running on GPU node

```
#!/bin/bash
#SBATCH -p gpu                          # Specify partition [Compute/Memory/GPU]
#SBATCH -N 1 –c 16                      # Specify number of nodes and processors per task
#SBATCH --gpus-per-node=4               # Specify number of GPU per task
#SBATCH --ntasks-per-node=4             # Specify tasks per node
#SBATCH -t 120:00:00                    # Specify maximum time limit (hour: minute: second)
#SBATCH -A tb901xxx                     # Specify project name
#SBATCH -J JOBNAME                      # Specify job name

module load Apptainer/1.1.6             # Load the Apptainer module

apptainer exec --nv -B $PWD:$PWD file.sif python3 file.py  # Run your program
```

Note: 1 GPU card: --gpus-per-node=1, 2 GPU cards: --gpus-per-node=2, 4 GPU cards: --gpus-per-node=4

Contact us

www.thaisc.io

thaisc-support@nstda.or.th